Taller Grupal Sockets Multihilo: Sistema de Chat Multiusuario en Red LAN por Consola

Multithread Sockets Group Workshop: Multiuser LAN Chat System via Console

Jairo Alejandro Ojeda
Líder del equipo y desarrollador del servidor
Carrera de Computación
Universidad Politécnica Salesiana del Ecuador – Sede Sur
Quito – Ecuador

Josué Samuel Vele
Desarrollador del cliente
Carrera de Computación
Universidad Politécnica Salesiana del Ecuador – Sede Sur
Quito – Ecuador

Juan David Figueroa
Documentador
Carrera de Computación
Universidad Politécnica Salesiana del Ecuador – Sede Sur
Quito – Ecuador

Asignatura: Sistemas Distribuidos · Docente: Ing. Ricardo Proaño
Quito – Ecuador, 16 de octubre del 2026

***Resumen*—Este trabajo documenta el diseño, la implementación y la validación de un sistema de chat multiusuario por consola para redes de área local (LAN), desarrollado en Python como evolución de un servidor de eco que atendía a un solo cliente y devolvía los mensajes únicamente a su emisor. Para superar esa limitación se implementó una arquitectura cliente-servidor sobre sockets TCP en la que el servidor, mediante un hilo por conexión, registra dinámicamente a los usuarios identificados con un alias y difunde (broadcast) cada mensaje a todos los participantes, mientras que el cliente emplea un hilo de recepción independiente para enviar y recibir de forma simultánea. La comunicación se rige por un protocolo de eventos JSON delimitados por salto de línea, y el código se estructuró aplicando principios SOLID, una clase abstracta (ABC) para el formato de mensajes y registros de eventos (logs). Las pruebas se ejecutaron en una red LAN real, con el servidor escuchando en 0.0.0.0:8080 y clientes en equipos distintos; se verificaron la conversación simultánea de hasta cuatro usuarios, el ingreso y la salida notificados, la reconexión con el mismo alias y la tolerancia a cierres imprevistos sin afectar al servidor, además de seis pruebas automatizadas superadas. El aporte del proyecto es una base modular, documentada y verificable para prácticas de programación concurrente y sistemas distribuidos.**

***Palabras clave*—Sockets, Multihilo, Chat LAN, TCP/IP, Programación Concurrente, SOLID, Python, Arquitectura Cliente-Servidor.**

I. INTRODUCCIÓN

A. Contextualización del problema

Los sistemas distribuidos se construyen sobre la comunicación entre procesos que se ejecutan en equipos diferentes, y los sockets constituyen la interfaz de programación más difundida para acceder a los servicios de transporte de la pila TCP/IP [1], [2], [3], [4]. En el taller de la asignatura se partió de un servidor de eco, es decir, un programa que acepta una conexión, lee los datos recibidos y los devuelve a quien los envió.

Este enfoque presenta cuatro limitaciones: (i) atiende de forma secuencial, por lo que un segundo cliente debe esperar a que el primero finalice; (ii) responde únicamente al emisor, de modo que no existe conversación entre personas; (iii) no identifica a los usuarios; y (iv) un fallo en la conexión de un cliente puede interrumpir el servicio completo.

B. Justificación

La comunicación en tiempo real entre varios participantes exige concurrencia: el servidor debe atender muchas conexiones a la vez y el cliente debe poder escribir mientras recibe mensajes de terceros [5], [6]. Resolver este problema en consola, sin interfaz gráfica, permite concentrar el aprendizaje en sockets, hilos, sincronización y manejo de errores, que son fundamentos de los sistemas distribuidos [4].

C. Objetivos

Objetivo general. Desarrollar e implementar un sistema de chat multiusuario por consola, basado en sockets TCP y multihilo en Python, que opere en una red LAN real y permita la conversación simultánea de varios usuarios identificados por alias.

Objetivos específicos:

1) Diseñar un servidor de difusión que registre dinámicamente a los clientes y atienda cada conexión en un hilo propio.
2) Implementar un cliente concurrente que envíe y reciba mensajes simultáneamente mediante hilos.
3) Definir un protocolo de aplicación que delimite los mensajes y permita identificar a los usuarios mediante alias.
4) Garantizar la robustez ante desconexiones abruptas y el cierre ordenado de los sockets.
5) Aplicar SOLID, clases abstractas y Clean Code, con registros de eventos que permitan auditar el sistema.
6) Validar el funcionamiento en una red LAN con hasta cuatro usuarios y documentar las evidencias, el control de versiones y el trabajo en equipo.

D. Estructura del documento

La Sección II revisa soluciones existentes de chat por consola. La Sección III presenta la metodología y la arquitectura, incluidos el protocolo y la aplicación de SOLID. La Sección IV describe la implementación y la configuración de las pruebas en LAN. La Sección V expone los resultados y las evidencias; la Sección VI discute las dificultades, las soluciones y las limitaciones; y la Sección VII formula las conclusiones y el trabajo futuro. Al final se incluyen las referencias y los anexos A al E.

II. TRABAJOS RELACIONADOS

Internet Relay Chat (IRC) es el referente histórico de los chats de texto en Internet: define un protocolo cliente-servidor en el que los servidores retransmiten los mensajes a los usuarios de un canal, identificados por un apodo [7]. Aunque IRC admite redes de servidores y canales, su complejidad excede el alcance de un taller; con el presente trabajo comparte la idea central de usuarios identificados y mensajes difundidos por un servidor.

Netcat permite abrir conexiones TCP arbitrarias desde la consola, pero en su uso clásico enlaza solo dos extremos. Ncat, su evolución dentro del proyecto Nmap, incorpora un modo de chat que retransmite lo que escribe cada cliente a los demás y los distingue con un identificador genérico [8]. Ambas herramientas resuelven el problema de forma genérica, sin validación de alias, protocolo propio, registros ni código que pueda estudiarse y extenderse.

En el ámbito académico, las guías de programación de sockets en Python presentan servidores de eco y chats con un hilo por cliente [9], [1]. Este trabajo parte de esos fundamentos y añade un protocolo de eventos delimitado por salto de línea, validación de alias y de longitud, separación de responsabilidades según SOLID [10], registros con rotación y pruebas automatizadas. Existen alternativas que escalan mejor a miles de conexiones, como la multiplexación de E/S con `selectors` o la programación asíncrona con `asyncio`; se eligió el modelo de un hilo por conexión por su claridad didáctica y por ser el objeto de estudio del taller.

III. METODOLOGÍA Y ARQUITECTURA

A. Organización del equipo y flujo de trabajo

El trabajo se organizó en tres roles con entregables diferenciados (Tabla I). Se empleó un flujo de ramas de Git inspirado en Git Flow: las ramas `feature/servidor`, `feature/cliente` y `feature/documentacion` se integran en `develop` y, una vez estabilizadas, en `main` (Anexo E). Durante las pruebas se utilizó además la rama `feature/logs-pruebas` para versionar los registros y la evidencia. El contrato de comunicación (Sección III-C) funciona como punto de integración entre servidor y cliente, lo que permite desarrollarlos en paralelo.

TABLA I
DIVISIÓN DEL TRABAJO Y ENTREGABLES

Integrante	Rol	Responsabilidad	Entregable
Jairo Alejandro Ojeda	Líder + desarrollador del servidor	Coordinación general; diseño e implementación del servidor de difusión	Servidor documentado, bitácora del líder, gestión de GitHub
Josué Samuel Vele	Desarrollador del cliente	Cliente concurrente con hilos (envío y recepción simultáneos)	Cliente documentado y evidencias de pruebas del cliente
Juan David Figueroa	Documentador	Documento IEEE, README visual, acta y consolidación de evidencias	README, documento IEEE, acta y control de versiones

B. Arquitectura cliente-servidor

El sistema sigue el modelo cliente-servidor sobre TCP [3], [11]. El servidor abre un socket de escucha en `0.0.0.0:8080`, lo que le permite aceptar conexiones por todas las interfaces del equipo anfitrión, y delega cada conexión aceptada en un hilo dedicado. Los clientes, en cualquier equipo de la LAN, se conectan a la dirección IP del anfitrión (Fig. 1). TCP ofrece un flujo de bytes fiable y ordenado, pero sin fronteras de mensaje [3]; por ello se definió un protocolo de aplicación (Sección III-C).

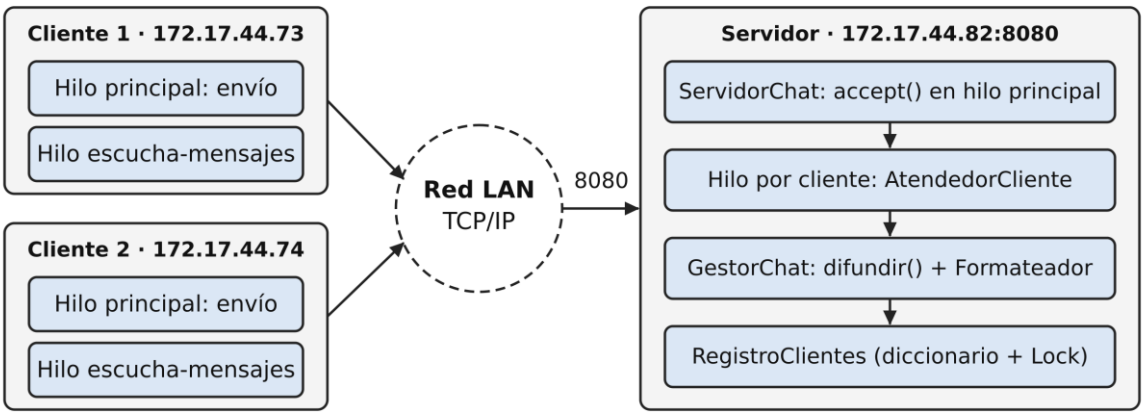


Fig. 1. Arquitectura cliente-servidor del chat multiusuario en la red LAN de pruebas.

C. Protocolo de aplicación

Cada evento se serializa como un objeto JSON [12] codificado en UTF-8 y terminado en salto de línea. El delimitador permite separar los mensajes, aunque TCP los agrupe o fragmente. Se establecen tres límites: 4096 bytes por evento, 500 caracteres por mensaje y 20 por alias, que protegen al servidor del consumo no controlado de memoria. La Tabla II resume los eventos y la Fig. 2 muestra una secuencia típica de ingreso, conversación y salida.

TABLA II
EVENTOS DEL PROTOCOLO DE APLICACIÓN

Evento	Sentido	Campos	Función
ingreso	Cliente → servidor	alias	Primer evento obligatorio de la sesión
mensaje	Cliente → servidor	texto	Mensaje escrito por el usuario
salida	Cliente → servidor	—	Cierre ordenado de la sesión
mensaje	Servidor → clientes	contenido	Texto formateado: [alias]: texto
sistema	Servidor → clientes	contenido	Avisos de ingreso y salida de usuarios
error	Servidor → cliente	contenido	Rechazo (alias inválido o repetido)

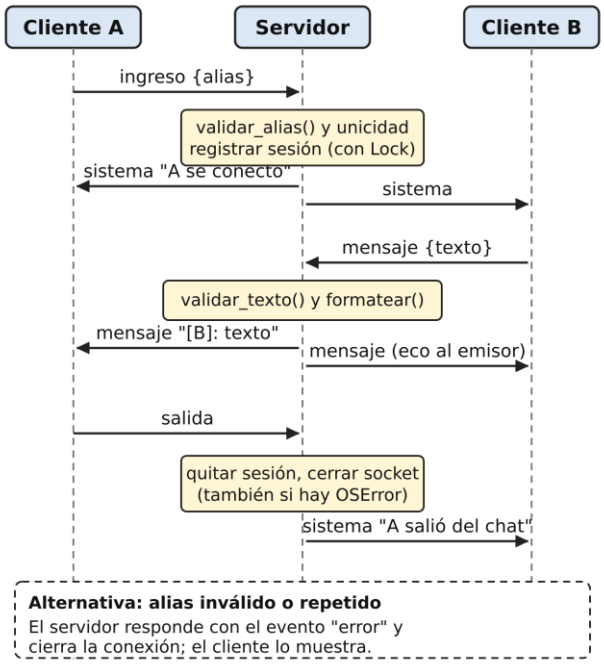


Fig. 2. Secuencia de eventos entre dos clientes y el servidor.

D. Servidor de difusión

El servidor se compone de cuatro clases con responsabilidades separadas (Fig. 3). `ServidorChat` crea el socket TCP, activa `SO_REUSEADDR`, escucha con una cola de 20 conexiones pendientes y, por cada conexión aceptada, lanza un hilo *daemon* de nombre `cliente-IP-puerto` [1], [5]. `AtendedorCliente` ejecuta en ese hilo el ciclo de vida del usuario: lee el evento de ingreso, registra la sesión, difunde el aviso de conexión, procesa sus mensajes y, al terminar, retira la sesión.

Registro dinámico de clientes. `RegistroClientes` mantiene un diccionario socket → `SesionCliente` protegido con un `threading.Lock`. El método `obtener_todos` devuelve una copia de la lista para que la difusión recorra las sesiones sin conservar el candado durante las operaciones de red, de modo que las altas y bajas no queden bloqueadas mientras se envían datos [6].

Difusión. `GestorChat.difundir` codifica el evento una sola vez y lo envía a cada sesión activa, incluido el emisor, de manera que todos los usuarios vean los mensajes en el mismo orden. `SesionCliente.enviar` utiliza un candado por socket para impedir que dos hilos mezclen bytes al escribir a la vez al mismo cliente. Si el envío falla, la sesión se cierra y el hilo propietario concluye.

E. Cliente concurrente

`ClienteChat` establece la conexión con `create_connection` (tiempo máximo de 10 s), envía el evento de ingreso y lanza un hilo *daemon* llamado `escucha-mensajes`, que ejecuta un bucle de recepción bloqueante. El hilo principal permanece atendiendo el teclado y envía cada línea escrita, de modo que la recepción y el envío ocurren simultáneamente [5]. Un `threading.Event` comunica a ambos hilos el estado de la sesión. La utilidad `patch_stdout` de `prompt_toolkit` [13] evita que los mensajes entrantes, impresos desde el hilo de escucha, desordenen el texto que el usuario está escribiendo. En la terminología del taller, el hilo principal cumple el rol de hilo de envío y `escucha-mensajes` el de hilo de recepción.

F. Identificación de usuarios

Todo cliente inicia la sesión con un evento `ingreso` que contiene su alias. El alias se valida (1 a 20 caracteres: letras, números, espacios, guion o guion bajo) en el cliente, antes de conectar, y de nuevo en el servidor. El servidor comprueba, dentro del candado del registro, que ningún usuario conectado tenga el mismo alias sin distinguir mayúsculas de minúsculas (`casefold`), con lo que la verificación y el alta son una operación atómica. Si el alias es inválido o está repetido, se responde con un evento `error` y se cierra la conexión. Cada mensaje se presenta con el formato `[alias]: texto`, por ejemplo `[Juan Figueroa]: Hola amigo`.

G. Manejo de excepciones y cierre limpio

`AtendedorCliente` encierra el ciclo de vida de cada cliente en un bloque `try/except/finally`. Los errores de red (`OSError`), de codificación (`UnicodeError`) y de protocolo (`ValueError`) se registran como advertencias y finalizan únicamente la conexión afectada; cualquier otra excepción se registra con su traza. En el bloque `finally` se retira la sesión del registro, se difunde "X salió del chat." y se cierra el socket, tanto en salidas ordenadas como en caídas abruptas (cierre de la consola, pérdida de red o cierre del proceso cliente). En el cliente, Ctrl+C y Ctrl+D se capturan para terminar enviando el evento `salida`, y `desconectar` ejecuta `shutdown` y `close`. El servidor termina de forma ordenada con Ctrl+C.

H. Aplicación de SOLID y de clases abstractas (ABC)

La Tabla III relaciona cada principio de diseño [10] con su aplicación en el código. `FormateadorMensaje` es una clase abstracta (módulo `abc` [14]) con el método abstracto `formatear`; `FormateadorConsola` la implementa y se inyecta en `GestorChat` desde `app_servidor.py`. Así, un nuevo formato (por ejemplo, con hora o colores) se incorpora creando otra subclase sin modificar el gestor. Como práctica de Clean Code [15] se emplean nombres descriptivos, funciones pequeñas, constantes con nombre (`MAX_TEXTO`, `MAX_ALIAS`) y registros en lugar de impresiones de depuración [16]. Una mejora pendiente es abstraer también `RegistroClientes` para completar la inversión de dependencias.

TABLA III
APLICACIÓN DE SOLID Y ABC EN EL CÓDIGO

Principio	Aplicación concreta
S: responsabilidad única	`RegistroClientes` guarda sesiones; `GestorChat` difunde; `AtendedorCliente` gestiona el ciclo de vida; `ServidorChat` solo acepta conexiones; `protocolo.py` y `registros.py` aíslan mensajes y logs
O: abierto/cerrado	Se añaden formatos de mensaje con nuevas subclases, sin modificar `GestorChat`
L: sustitución de Liskov	`FormateadorConsola` sustituye a `FormateadorMensaje` sin alterar el comportamiento esperado
I: segregación de interfaces	La interfaz abstracta expone un solo método: `formatear(alias, texto)`
D: inversión de dependencias	`GestorChat` depende de la abstracción `FormateadorMensaje`, inyectada por constructor
ABC	`FormateadorMensaje(ABC)` con `@abstractmethod`

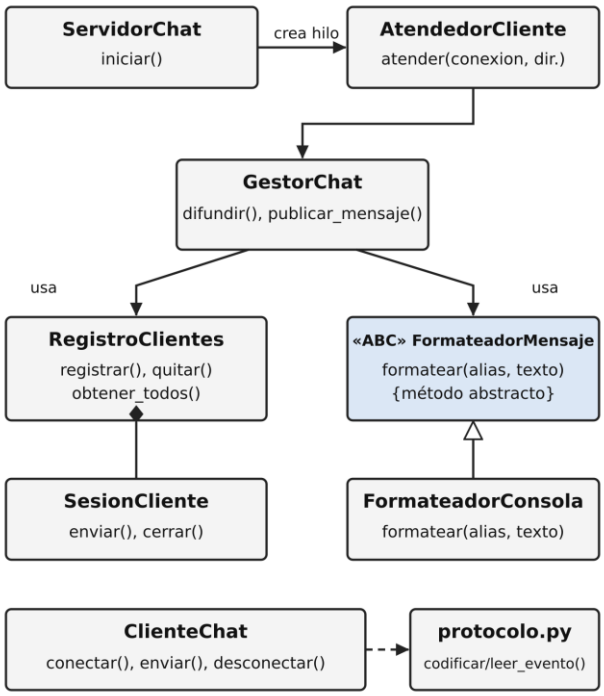


Fig. 3. Diagrama de clases del servidor, del cliente y del módulo de protocolo.

IV. DESARROLLO E IMPLEMENTACIÓN

A. Lenguaje y herramientas

Se utilizó Python 3 (versión 3.14.5 en los entornos virtuales de las evidencias); el código requiere 3.10 o superior por el uso de anotaciones de unión como `dict | None`. Se emplearon los módulos estándar `socket`, `threading`, `json`, `re`, `logging`, `argparse`, `dataclasses`, `abc`, `sys` y `pathlib` [16], y la biblioteca externa `prompt_toolkit` 3.x solo en el cliente. Como herramientas de apoyo se usaron PyCharm, Git y GitHub, Windows PowerShell y el módulo `unittest` [17].

B. Estructura del código

La Tabla IV lista los archivos del proyecto. Los puntos de entrada se denominan `app_servidor.py` y `app_cliente.py`, y la lógica reside en el paquete `chat`; por tanto, lo que el enunciado denomina `servidor.py` y `cliente.py` corresponde a `chat/servidor.py` y `chat/cliente.py` junto con sus puntos de entrada.

TABLA IV
ARCHIVOS DEL PROYECTO Y RESPONSABLES

Archivo	Responsable	Función
app_servidor.py	Jairo A. Ojeda	Punto de entrada: argumentos `--host` y `--puerto`
chat/servidor.py	Jairo A. Ojeda	Sesiones, registro, difusión y atención por hilos
chat/formato.py	Jairo A. Ojeda	Clase abstracta de formato y su implementación
app_cliente.py	Josué S. Vele	Punto de entrada: alias, entrada con prompt_toolkit
chat/cliente.py	Josué S. Vele	Cliente con hilo de escucha y envío
chat/protocolo.py	Equipo	Eventos JSON, validación de alias y texto
chat/registros.py	Equipo	Logs en consola y archivo rotativo
pruebas/test_chat.py	Equipo	3 pruebas unitarias y 3 de integración
README.md	Juan D. Figueroa	Guía visual de uso (Anexo B)

C. Servidor (responsable: Jairo Alejandro Ojeda)

El ciclo de aceptación crea un hilo por conexión. Los fragmentos siguientes se reformatearon por el ancho de columna; el código completo está en el Anexo A.

```
# chat/servidor.py - ServidorChat.iniciar (extracto)
servidor.setsockopt(socket.SOL_SOCKET,
                    socket.SO_REUSEADDR, 1)
servidor.bind((self._host, self._puerto))
servidor.listen(20)
while True:
    conexion, direccion = servidor.accept()
    hilo = threading.Thread(
        target=self._atendedor.atender,
        args=(conexion, direccion),
        name=f"cliente-{direccion[0]}-{direccion[1]}",
        daemon=True)
    hilo.start()
```

El registro comprueba la unicidad del alias dentro del candado, por lo que dos clientes que ingresan al mismo tiempo no pueden obtener el mismo nombre.

```
# chat/servidor.py - RegistroClientes.registrar
def registrar(self, sesion: SesionCliente) -> None:
    with self._candado:
        if any(otra.alias.casefold() ==
               sesion.alias.casefold()
               for otra in self._sesiones.values()):
            raise ValueError("Ese alias ya está "
                             "conectado. Escoge otro.")
        self._sesiones[sesion.conexion] = sesion
```

La difusión envía el mismo paquete a todas las sesiones; ante un fallo de envío cierra la sesión afectada sin interrumpir a las demás.

```
# chat/servidor.py - GestorChat.difundir
def difundir(self, tipo: str,
             contenido: str) -> None:
    paquete = codificar_evento(
        tipo, contenido=contenido)
    for sesion in self._registro.obtener_todos():
        try:
            sesion.enviar(paquete)
        except OSError as error:
            REGISTRO.warning("Falló el envío a %s: %s",
                             sesion.alias, error)
            sesion.cerrar()
```

Por último, el ciclo de vida de cada cliente garantiza la limpieza en el bloque `finally`, incluso ante desconexiones abruptas.

```
# chat/servidor.py - AtendedorCliente.atender
try:
    with conexion, conexion.makefile(
            "r", encoding="utf-8") as flujo:
        ingreso = leer_evento(flujo)
        ...  # valida alias, registra y avisa
        while True:
            evento = leer_evento(flujo)
            if evento is None or \
                    evento["tipo"] == "salida":
                break
            texto = validar_texto(evento.get("texto"))
            self._gestor.publicar_mensaje(alias, texto)
except (OSError, UnicodeError, ValueError) as error:
    REGISTRO.warning("Conexión %s:%s finalizó "
                     "con aviso: %s", ...)
finally:
    if sesion is not None and \
            self._gestor.registro.quitar(sesion):
        self._gestor.difundir("sistema",
            f"{sesion.alias} salió del chat.")
    conexion.close()
```

D. Cliente (responsable: Josué Samuel Vele)

El cliente conecta, envía su alias y lanza el hilo de recepción; el tiempo de espera de 10 s se desactiva una vez establecida la conexión para que la lectura no expire por inactividad.

```
# chat/cliente.py - ClienteChat.conectar
```

```
self._conexion = socket.create_connection(
    (self._host, self._puerto), timeout=10)
self._conexion.settimeout(None)
self._activo.set()
self._conexion.sendall(
    codificar_evento("ingreso", alias=alias))
self._hilo_escucha = threading.Thread(
    target=self._escuchar,
    name="escucha-mensajes", daemon=True)
self._hilo_escucha.start()
```

El hilo de recepción imprime los eventos `mensaje` y `sistema` y termina cuando el servidor cierra la conexión.

```
# chat/cliente.py - ClienteChat._escuchar
with self._conexion.makefile(
        "r", encoding="utf-8") as flujo:
    while self._activo.is_set():
        evento = leer_evento(flujo)
        if evento is None:
            print("\n[INFO] El servidor cerró "
                  "la conexión.")
            break
        if evento["tipo"] in {"mensaje", "sistema"}:
            print(f"\n{evento.get('contenido', '')}")
```

El hilo principal lee el teclado con `prompt_toolkit`; `patch_stdout` mantiene intacta la línea en edición cuando llegan mensajes.

```
# app_cliente.py - bucle de entrada (extracto)
with patch_stdout():
    while cliente.esta_activo():
        texto = sesion.prompt("Tú > ")
        if texto.strip().lower() == "/salir":
            break
        if not texto.strip():
            continue
        cliente.enviar(texto)
```

E. Documentación y README (responsable: Juan David Figueroa)

El documentador consolidó la evidencia de las pruebas (capturas de terminal y archivos `logs/servidor.log` y `logs/cliente.log`), ordenó cronológicamente las capturas según sus marcas de tiempo y redactó el README con insignias, tablas, diagramas ASCII y una guía de conexión en LAN (Anexo B), así como el acta de reunión (Anexo C) y la estrategia de control de versiones (Anexo E). El borrador de la bitácora del líder se incluye en el Anexo D.

F. Configuración de las pruebas en LAN

Las pruebas se realizaron en una red local con el direccionamiento de la Tabla V. El servidor escucha en `0.0.0.0` para aceptar conexiones desde cualquier interfaz del anfitrión; los clientes indican la IP real del anfitrión mediante `--host`. La conectividad se verificó previamente con `Test-NetConnection` [18].

TABLA V
PARÁMETROS DE RED DE LA PRUEBA

Elemento	Valor
Servidor (anfitrión)	172.17.44.82, puerto TCP 8080, escucha en 0.0.0.0
Cliente 1	172.17.44.73 (equipo de Juan David Figueroa)
Cliente 2	172.17.44.74 (equipo de Josué Samuel Vele)
Cliente local	127.0.0.1 (en el equipo del servidor)
Comando del servidor	`py app_servidor.py --host 0.0.0.0 --puerto 8080`
Comando del cliente	`python app_cliente.py --host 172.17.44.82 --puerto 8080`

Pasos para conectar desde otra máquina o máquina virtual:

1) Obtener la IPv4 del anfitrión con `ipconfig` y confirmar que ambos equipos pertenecen a la misma red.
2) Iniciar el servidor con `--host 0.0.0.0` y comprobar el registro “Servidor de chat activo”.
3) Permitir el puerto 8080 en el cortafuegos del anfitrión si bloquea las conexiones entrantes.
4) Desde el cliente, verificar con `Test-NetConnection 172.17.44.82 -Port 8080` que `TcpTestSucceeded` sea `True`.
5) Ejecutar el cliente con la IP del anfitrión e ingresar el alias.
6) Conversar y salir con `/salir`. En una máquina virtual, configurar el adaptador en modo puente para que obtenga una IP de la LAN.

V. RESULTADOS Y EVIDENCIAS

Las pruebas se realizaron el 16 de octubre del 2026 entre las 17:24 y las 17:41, con el servidor en el equipo del líder (172.17.44.82) y clientes en los equipos 172.17.44.73 y 172.17.44.74 (Tabla V). Las capturas se presentan en orden cronológico según las marcas de tiempo de los registros y cada figura incluye su pie en formato IEEE (“Fig. N.”, centrado bajo la imagen).

A. Inicio del servidor y primer usuario local

La Fig. 4 muestra la terminal del equipo anfitrión. El servidor se inició con `py app_servidor.py --host 0.0.0.0 --puerto 8080` y registró “Servidor de chat activo en 0.0.0.0:8080” a las 17:25:29. Cada línea del registro contiene fecha y hora, nivel, nombre del hilo (por ejemplo, `cliente-127.0.0.1-54644`), módulo y mensaje, lo que permite rastrear qué hilo atiende cada conexión. El primer usuario, con alias “Alejo” desde 127.0.0.1, ingresó a las 17:28:31 y difundió un mensaje de 22 caracteres. Se aprecia que el servidor registra la longitud del mensaje y no su contenido.

B. Verificación de la conectividad

En la Fig. 5a se observa un primer intento de conexión desde el equipo 172.17.44.73 a las 17:24:12, anterior al arranque del servidor registrado a las 17:25:29. Terminó con el error “No fue posible conectarse: timed out”, la recomendación “Comprueba la IP, el puerto, el firewall y que el servidor esté iniciado.” y el mensaje “Chat finalizado.”, sin excepciones sin controlar. Posteriormente, `Test-NetConnection 172.17.44.82 -Port 8080` devolvió `TcpTestSucceeded : True` desde ambos equipos (Fig. 5a y 5b), con direcciones de origen 172.17.44.73 y 172.17.44.74. En el servidor, conexiones de este tipo, que se cierran sin enviar el evento de ingreso, quedan registradas con la advertencia “El primer evento debe indicar el alias del usuario.” (Fig. 4, 17:29:29 a 17:31:39), sin afectar al servicio.

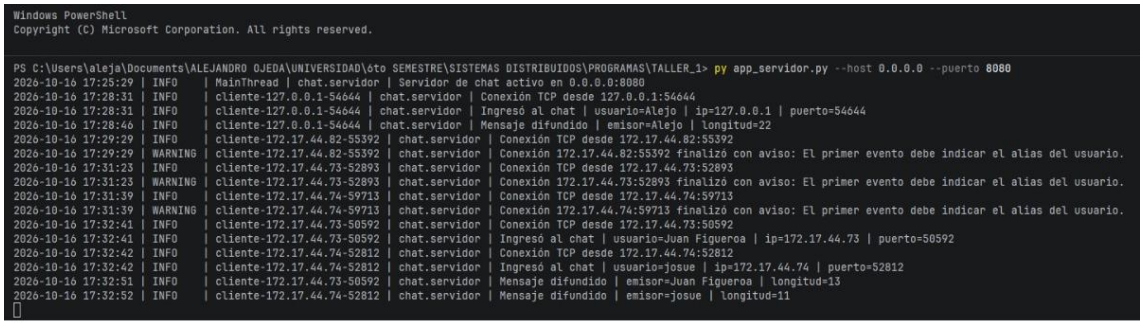


Fig. 4. Terminal del servidor en el equipo anfitrión: inicio, ingreso del usuario local y avisos por conexiones sin alias.

(a) Equipo de Juan David Figueroa (172.17.44.73)



(b) Equipo de Josué Samuel Vele (172.17.44.74)



Fig. 5. Verificación de la conectividad TCP hacia 172.17.44.82:8080: (a) equipo 172.17.44.73, con un intento fallido previo; (b) equipo 172.17.44.74.

C. Primera conversación entre dos clientes remotos

A las 17:32:36 y 17:32:37 se conectaron “Juan Figueroa” (172.17.44.73) y “josue” (172.17.44.74). La Fig. 6 muestra que cada cliente recibe el aviso de ingreso del otro y que los mensajes aparecen con el formato `[alias]: texto` en ambas terminales, incluida la del emisor, que recibe el eco de su propio mensaje. La línea de entrada “Tú >” permanece al final gracias a `patch_stdout`. El servidor registró ambos mensajes a las 17:32:51 y 17:32:52, con longitudes de 13 y 11 caracteres.

D. Salida de usuarios y comando de salida

En la Fig. 7a, “Josue Vele” envió el texto `exit` a las 17:36:22. Como el comando de salida del sistema es `/salir`, el texto se difundió como un mensaje ordinario (`[Josue Vele]: exit`); después la sesión local se cerró (“Conexión local cerrada.”, “Hilo de recepción terminado.”). En la otra terminal (Fig. 7b) se recibió “Josue Vele salió del chat.” y el usuario terminó con `/salir`. En esta última captura aparece, tras “Chat finalizado.”, un aviso de Python (`Fatal Python error: _enter_buffered_busy`) producido al finalizar el intérprete; no afectó al servidor ni a otros clientes y se analiza en la Sección VI.

(a) Cliente "Juan Figueroa" (172.17.44.73)

```
(.venv) PS C:\Users\USUARIO\PycharmProjects\Taller-Grupal-Sockets-Multihilo> python app_cliente.py --host 172.17.44.82 --puerto 8080
Tu nombre para el chat: Juan Figueroa
2026-10-16 17:32:36 | INFO     | MainThread | chat.cliente | Conectado al servidor 172.17.44.82:8080 como Juan Figueroa

Juan Figueroa se conectó al chat.
Conectado. Escribe y presiona Enter. Para salir: /salir

josue se conectó al chat.
Tú > Hola amikoooo
2026-10-16 17:32:46 | INFO     | MainThread | chat.cliente | Mensaje enviado | longitud=13

[Juan Figueroa]: Hola amikoooo

[josue]: hola mi bro
Tú >
```

(b) Cliente "josue" (172.17.44.74)

```
(.venv) PS C:\Users\josue\PycharmProjects\Taller-Grupal-Sockets-Multihilo> python app_cliente.py --host 172.17.44.82 --puerto 8080
Tu nombre para el chat: josue
2026-10-16 17:32:37 | INFO     | MainThread | chat.cliente | Conectado al servidor 172.17.44.82:8080 como josue
Conectado. Escribe y presiona Enter. Para salir: /salir

josue se conectó al chat.

[Juan Figueroa]: Hola amikoooo
Tú > hola mi bro
2026-10-16 17:32:46 | INFO     | MainThread | chat.cliente | Mensaje enviado | longitud=11

[josue]: hola mi bro
Tú >
```

Fig. 6. Primera conversación entre dos clientes remotos: (a) “Juan Figueroa”; (b) “josue”.

(a) Josué: el texto "exit" se difunde como mensaje; cierre de la sesión

```
Josue Vele se conectó al chat.
Tú > Hola
2026-10-16 17:35:58 | INFO     | MainThread | chat.cliente | Mensaje enviado | longitud=4

[Josue Vele]: Hola

[Juan]: Hola amikooooo
Tú > exit
2026-10-16 17:36:22 | INFO     | MainThread | chat.cliente | Mensaje enviado | longitud=4

[Josue Vele]: exit
Tú >
2026-10-16 17:37:01 | INFO     | MainThread | chat.cliente | Conexión local cerrada.
Chat finalizado.

[INFO] El servidor cerró la conexión.
2026-10-16 17:37:01 | INFO     | escucha-mensajes | chat.cliente | Hilo de recepción terminado.
(.venv) PS C:\Users\josue\PycharmProjects\Taller-Grupal-Sockets-Multihilo> python app_cliente.py --host 172.17.44.82 --puerto 8080
Tu nombre para el chat:
```

(b) Juan: recibe "salió del chat" y cierra con /salir

```
[Juan]: Hola amikooooo

[Josue Vele]: exit

Josue Vele salió del chat.
Tú > /salir
2026-10-16 17:37:08 | INFO     | MainThread | chat.cliente | Conexión local cerrada.
Chat finalizado.

[INFO] El servidor cerró la conexión.Fatal Python error: _enter_buffered_busy: could not acquire lock for <_io.BufferedWriter name='<stdout>'> at interpreter shutdown, possibly due
 to daemon threads
Python runtime state: finalizing (tstate=0x00007ffdfa735d30)

Current thread 0x000019f4 (most recent call first):
  <no Python frame>

Extension modules: wcwidth._wcwidth_c (total: 1)
(.venv) PS C:\Users\USUARIO\PycharmProjects\Taller-Grupal-Sockets-Multihilo> python app_cliente.py --host 172.17.44.82 --puerto 8080
Tu nombre para el chat:
```

Fig. 7. Comportamiento del texto “exit” y cierre de sesión con /salir: (a) terminal de Josué; (b) terminal de Juan.

E. Reconexión con el mismo alias

Tras salir, los usuarios volvieron a conectarse con los alias “Josue Vele” (17:38:54) y “Juan Figueroa” (17:38:43), como muestra la Fig. 8. El registro del servidor confirma que “Josue Vele” había salido a las 17:37:06 y reingresó a las 17:38:59, por lo que el alias se libera al cerrar la sesión y puede reutilizarse (Fig. 11). Los usuarios intercambiaron mensajes a las 17:38:48, 17:39:01 y 17:39:08, y “Josue Vele” abandonó el chat con `/salir` a las 17:39:17.

F. Participación del cliente del anfitrión

El equipo anfitrión ejecutó también un cliente local. El archivo `cliente.log` (Fig. 9a) registra las sesiones de “Alejo” (17:28:31) y “Alejandro” (17:34:55) hacia 127.0.0.1:8080 y los mensajes enviados (22, 15 y 21 caracteres). La terminal del anfitrión (Fig. 9b) recibe toda la conversación de los clientes remotos, y el mensaje que “Alejandro” envió a las 17:39:25 llegó al equipo de Juan (Fig. 9c), lo que demuestra la difusión bidireccional entre el anfitrión y los demás equipos.

(a) Josué: reconexión con el alias "Josue Vele"

```
(.venv) PS C:\Users\josue\PycharmProjects\Taller-Grupal-Sockets-Multihilo> python app_cliente.py --host 172.17.44.82 --puerto 8080
Tu nombre para el chat: Josue Vele
2026-10-16 17:38:54 | INFO    | MainThread | chat.cliente | Conectado al servidor 172.17.44.82:8080 como Josue Vele
Conectado. Escribe y presiona Enter. Para salir: /salir

Josue Vele se conectó al chat.
Tú > Hola
2026-10-16 17:39:01 | INFO    | MainThread | chat.cliente | Mensaje enviado | longitud=4

[Josue Vele]: Hola

[Juan Figueroa]: Hola amikooo
Tú > /salir
2026-10-16 17:39:17 | INFO    | MainThread | chat.cliente | Conexión local cerrada.
Chat finalizado.

[INFO] El servidor cerró la conexión.
2026-10-16 17:39:17 | INFO    | escucha-mensajes | chat.cliente | Hilo de recepción terminado.
```

(b) Juan: reconexión con el alias "Juan Figueroa"

```
(.venv) PS C:\Users\USUARIO\PycharmProjects\Taller-Grupal-Sockets-Multihilo> python app_cliente.py --host 172.17.44.82 --puerto 8080
Tu nombre para el chat: Juan Figueroa
2026-10-16 17:38:43 | INFO    | MainThread | chat.cliente | Conectado al servidor 172.17.44.82:8080 como Juan Figueroa
Conectado. Escribe y presiona Enter. Para salir: /salir

Juan Figueroa se conectó al chat.
Tú > Hola amigo
2026-10-16 17:38:48 | INFO    | MainThread | chat.cliente | Mensaje enviado | longitud=10

[Juan Figueroa]: Hola amigo

Josue Vele se conectó al chat.

[Josue Vele]: Hola
Tú > Hola amikooo
2026-10-16 17:39:08 | INFO    | MainThread | chat.cliente | Mensaje enviado | longitud=12

[Juan Figueroa]: Hola amikooo
Tú >
```

Fig. 8. Reconexión con el mismo alias tras salir del chat: (a) Josué; (b) Juan.

(a) cliente.log del anfitrión (127.0.0.1:8080)

app_cliente.py .gitignore cliente.log servidor.log

```
2026-10-16 17:28:31 | INFO    | MainThread | chat.cliente | Conectado al servidor 127.0.0.1:8080 como Alejo
2026-10-16 17:28:46 | INFO    | MainThread | chat.cliente | Mensaje enviado | longitud=22
2026-10-16 17:34:55 | INFO    | MainThread | chat.cliente | Conectado al servidor 127.0.0.1:8080 como Alejandro
2026-10-16 17:34:59 | INFO    | MainThread | chat.cliente | Mensaje enviado | longitud=15
2026-10-16 17:39:25 | INFO    | MainThread | chat.cliente | Mensaje enviado | longitud=21
```

(b) Terminal del anfitrión: recibe toda la conversación

```
[Juan]: Hola amikooooo

[Josue Vele]: exit

Josue Vele salió del chat.

Juan salió del chat.

Juan Figueroa se conectó al chat.

[Juan Figueroa]: Hola amigo

Josue Vele se conectó al chat.

[Josue Vele]: Hola

[Juan Figueroa]: Hola amikooo

Josue Vele salió del chat.
Tú > Hola amigooooooooooos
2026-10-16 17:39:25 | INFO    | MainThread | chat.cliente | Mensaje enviado | longitud=21

[Alejandro]: Hola amigooooooooooos

Juan Figueroa salió del chat.
Tú >
```

(c) Equipo de Juan: recibe "[Alejandro]: Hola amigooo..."

```
Juan Figueroa se conectó al chat.
Tú > Hola amigo
2026-10-16 17:38:48 | INFO    | MainThread | chat.cliente | Mensaje enviado | longitud=10

[Juan Figueroa]: Hola amigo

Josue Vele se conectó al chat.

[Josue Vele]: Hola
Tú > Hola amikooo
2026-10-16 17:39:08 | INFO    | MainThread | chat.cliente | Mensaje enviado | longitud=12

[Juan Figueroa]: Hola amikooo

Josue Vele salió del chat.

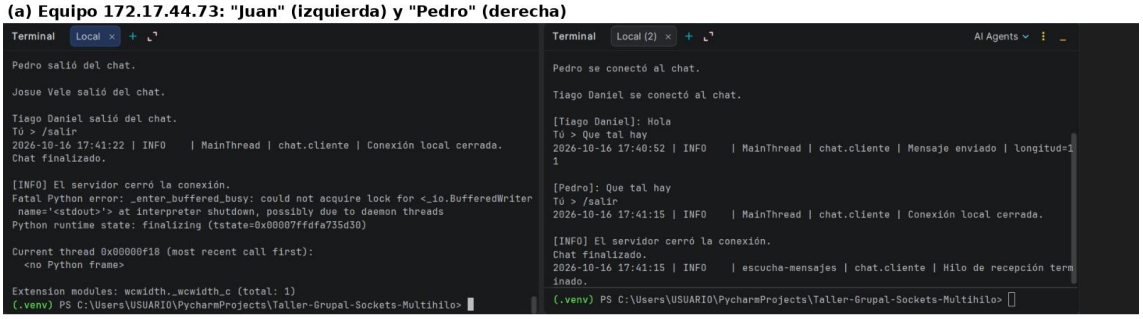
[Alejandro]: Hola amigooooooooooos
Tú >
```

Fig. 9. Participación del cliente local del anfitrión: (a) cliente.log; (b) terminal del anfitrión; (c) mensaje recibido en el equipo de Juan.

G. Prueba con cuatro usuarios simultáneos

Entre las 17:40:07 y las 17:41:27 se conectaron cuatro usuarios al mismo tiempo desde dos equipos, con dos procesos cliente por equipo: “Juan” y “Pedro” desde 172.17.44.73, y “Josue Vele” y “Tiago Daniel” desde 172.17.44.74 (Fig. 10). El servidor atendió cada conexión en un hilo propio (puertos de origen distintos en la Fig. 11). Los cuatro recibieron los mensajes de “Tiago Daniel” (17:40:52) y de “Pedro” (17:40:57), y las salidas se notificaron una a una: Pedro a las 17:41:20, Josue Vele a las 17:41:21, Tiago Daniel a las 17:41:25 y Juan a las 17:41:27, sin afectar a los usuarios que permanecían conectados. Se observa un desfase de unos cinco segundos entre las marcas de tiempo de los clientes y las del servidor (por ejemplo, la salida de Pedro figura a las 17:41:15 en su cliente), atribuible a relojes no sincronizados entre los equipos.

(a) Equipo 172.17.44.73: "Juan" (izquierda) y "Pedro" (derecha)



(b) Equipo 172.17.44.74: "Tiago Daniel" (arriba) y "Josue Vele" (abajo)



Fig. 10. Prueba con cuatro usuarios simultáneos desde dos equipos de la LAN: (a) equipo 172.17.44.73; (b) equipo 172.17.44.74.

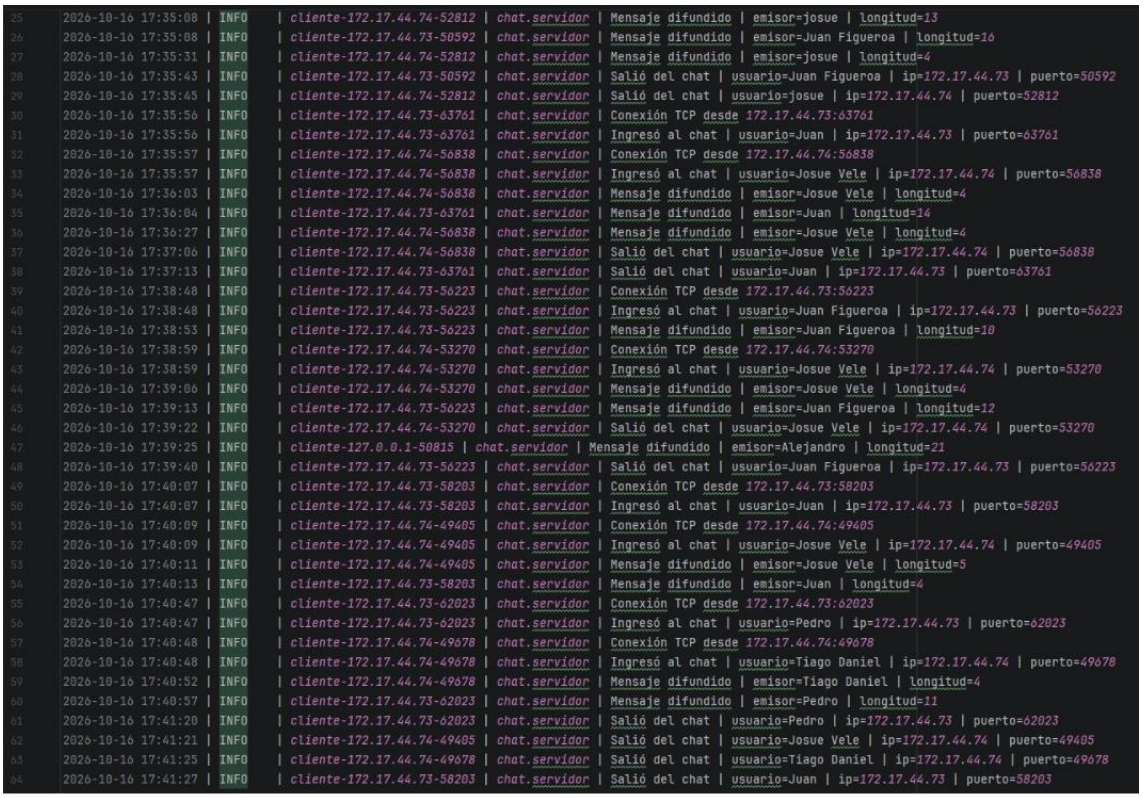


Fig. 11. Archivo servidor.log durante las pruebas (17:35:08 a 17:41:27): ingresos, mensajes y salidas de los usuarios.

H. Casos de prueba y pruebas automatizadas

La Tabla VI contrasta el resultado esperado con el obtenido en cada caso. Los casos C13 a C15 corresponden a la suite automatizada `pruebas/test_chat.py`, que contiene tres pruebas unitarias y tres de integración con sockets reales sobre `localhost` [17]; al ejecutarla con

`python -m unittest -v pruebas.test_chat` las seis pruebas se superaron.

TABLA VI
CASOS DE PRUEBA: RESULTADO ESPERADO FRENTE A OBTENIDO

ID	Caso de prueba	Resultado esperado	Resultado obtenido	Evidencia
C1	Inicio del servidor en 0.0.0.0:8080	El servidor escucha y lo registra	"Servidor de chat activo en 0.0.0.0:8080" (17:25:29)	Fig. 4
C2	Ingreso de un cliente local con alias y envío de un mensaje	Sesión registrada y mensaje difundido	"Alejo" ingresó (17:28:31) y difundió 22 caracteres	Fig. 4, 9
C3	Conectividad TCP desde otros equipos	TcpTestSucceeded igual a True	True desde 172.17.44.73 y .74; el servidor registró advertencias sin colapsar	Fig. 4, 5
C4	Conexión con servidor no disponible	Mensaje orientativo y cierre limpio del cliente	"timed out", recomendación y "Chat finalizado."	Fig. 5a
C5	Conversación entre dos clientes remotos	Ambos reciben los mensajes con alias	[Juan Figueroa] y [josue] visibles en ambos clientes	Fig. 6
C6	Avisos de ingreso y salida	"X se conectó/salió del chat." para todos	Avisos recibidos en todas las terminales	Fig. 6 a 8
C7	Texto "exit" (no es un comando)	Se difunde como mensaje ordinario	"[Josue Vele]: exit" recibido por todos	Fig. 7a
C8	Salida ordenada con /salir	Cierra la sesión; el resto continúa; el servidor registra la salida	"Conexión local cerrada." y "Salió del chat" en el servidor	Fig. 7b, 11
C9	Reconexión con el mismo alias tras salir	El alias se libera y se reutiliza	Josue Vele salió 17:37:06 y reingresó 17:38:59	Fig. 8, 11
C10	Cliente del anfitrión junto a clientes remotos	Envía y recibe en toda la LAN	"[Alejandro]" recibido en el equipo de Juan	Fig. 9
C11	Cuatro usuarios simultáneos desde dos equipos	Difusión a los cuatro usuarios	Juan, Pedro, Josue Vele y Tiago Daniel conversan	Fig. 10, 11
C12	Salida escalonada de los cuatro usuarios	Cada salida se notifica y el servidor sigue activo	Salidas a las 17:41:20, :21, :25 y :27	Fig. 10, 11
C13	Alias repetido (automatizada)	Evento "error" y rechazo	Superada: test_alias_duplicado_se_rechaza	Suite
C14	Cierre imprevisto de un cliente (automatizada)	El servidor notifica la salida y continúa	Superada: test_cierre_imprevisto_de_un_cliente	Suite
C15	Suite completa de `unittest`	Seis pruebas superadas	Seis pruebas superadas (OK)	Suite

VI. DISCUSIÓN

A. Dificultades encontradas y soluciones aplicadas

1) **Concurrencia y condiciones de carrera.** El diccionario de sesiones se modifica desde varios hilos y un mismo socket puede recibir escrituras simultáneas. Se aplicaron un candado en `RegistroClientes` y otro por sesión (`candado_envio`), y `obtener_todos` entrega una copia para no retener el candado durante la E/S [5], [6].
2) **Fronteras de mensaje en TCP.** El flujo de bytes no conserva los límites de cada envío [3]. Se adoptó un protocolo de una línea JSON por evento con lectura acotada (`readline` con tope), verificado por la prueba de dos eventos consecutivos.
3) **Bloqueos.** `accept()` y la lectura de sockets bloquean el hilo que las invoca. Se resolvió con un hilo por conexión en el servidor y un hilo *daemon* de escucha en el cliente, y con un tiempo máximo de 10 s solo para establecer la conexión.
4) **Entrada de consola frente a mensajes entrantes.** Los mensajes impresos por el hilo de escucha interrumpían el texto que el usuario escribía; se resolvió con `prompt_toolkit` y `patch_stdout` [13].
5) **Alias duplicados.** Verificar y registrar por separado habría permitido duplicados con ingresos simultáneos; la comprobación se hace dentro del candado del registro, sin distinguir mayúsculas.
6) **Desconexiones abruptas.** Se capturan `OSError`, `UnicodeError` y `ValueError` por conexión y el bloque `finally` retira la sesión, avisa a los demás y cierra el socket (casos C13 y C14).
7) **Conectividad en la LAN.** El intento fallido de la Fig. 5a mostró que el cliente debe orientar al usuario ante un tiempo de espera agotado. Se incorporó el mensaje sobre IP, puerto, cortafuegos y servidor iniciado, y se adoptó `Test-NetConnection` como verificación previa [18].
8) **Aviso al finalizar el cliente (hallazgo abierto).** En las capturas de las Fig. 7b y 10a, tras "Chat finalizado.", Python 3.14.5 emite `Fatal Python error: _enter_buffered_busy`, con la nota "possibly due to daemon threads": el hilo *daemon* de recepción intenta escribir en la salida mientras el intérprete finaliza. No afecta al servidor ni a la conversación, pero debe corregirse esperando al hilo (`join` con tiempo máximo) en `desconectar`.

B. Comparación con el código base original (servidor de eco)

La Tabla VII contrasta el servidor de eco descrito en el enunciado del taller con el sistema desarrollado.

TABLA VII
SERVIDOR DE ECO FRENTE AL CHAT MULTIUSUARIO

Aspecto	Servidor de eco	Chat desarrollado
Concurrencia	Un cliente a la vez	Un hilo por cliente
Destinatarios	Solo el emisor	Todos los usuarios activos
Identificación	Ninguna	Alias validado y único
Protocolo	Bytes sin estructura	Eventos JSON por línea con límites
Robustez	Un fallo puede detener el servicio	Fallos aislados por hilo y limpieza en `finally`

Aspecto	Servidor de eco	Chat desarrollado
Observabilidad	Impresiones en consola	Logs con hilo y rotación de archivos
Verificación	Manual	Seis pruebas automáticas y prueba en LAN

C. Limitaciones del sistema

1) La comunicación viaja en texto plano y no existe autenticación; solo es apropiada para redes de confianza.
2) El modelo de un hilo por conexión no escala a miles de usuarios; las alternativas son `selectors` o `asyncio`.
3) La difusión es secuencial y `sendall` es bloqueante: un cliente lento puede retrasar la entrega a los siguientes.
4) No hay historial de mensajes, mensajes privados ni salas, y el servidor no limita el número de clientes.
5) Las marcas de tiempo de los registros dependen de relojes no sincronizados entre equipos.
6) El aviso de Python al cerrar el cliente (Sección VI-A) está pendiente de corrección.

VII. CONCLUSIONES

A. Logros alcanzados

Se construyó un chat multiusuario por consola que funciona en una red LAN real: el servidor atiende a varios clientes concurrentes mediante hilos, difunde cada mensaje a todos los usuarios identificados por alias y se mantiene estable ante salidas ordenadas y cierres imprevistos. La prueba con cuatro usuarios desde dos equipos y la suite de seis pruebas automatizadas respaldan el cumplimiento de los objetivos específicos. La separación en módulos, la clase abstracta de formato y los registros de eventos dejan una base mantenible.

B. Aprendizajes del equipo

El equipo comprobó que TCP no preserva fronteras de mensaje y que la concurrencia exige sincronizar los recursos compartidos; que los registros por hilo facilitan el diagnóstico en entornos distribuidos; que la conectividad en LAN (dirección de escucha, cortafuegos, relojes) condiciona tanto como el código; y que dividir roles y trabajar con ramas facilita la integración.

C. Trabajo futuro

Se propone: (i) esperar al hilo de recepción al cerrar el cliente para eliminar el aviso final; (ii) cifrar la comunicación con TLS y autenticar usuarios; (iii) incorporar comandos como `/usuarios` y mensajes privados; (iv) añadir salas de conversación; (v) limitar el tiempo de envío por cliente; (vi) evaluar una versión con `asyncio` y pruebas de carga; y (vii) almacenar un historial de mensajes.

REFERENCIAS

[1] Python Software Foundation, “socket — Low-level networking interface,” *Python 3 Documentation*. [En línea]. Disponible: https://docs.python.org/3/library/socket.html

[2] IEEE and The Open Group, *IEEE Std 1003.1-2017 (Revision of IEEE Std 1003.1-2008), Standard for Information Technology—Portable Operating System Interface (POSIX)*, 2018.

[3] W. Eddy, Ed., “Transmission Control Protocol (TCP),” IETF, RFC 9293, ago. 2022. [En línea]. Disponible: https://www.rfc-editor.org/info/rfc9293

[4] A. S. Tanenbaum y M. van Steen, *Distributed Systems: Principles and Paradigms*, 2.ª ed. Upper Saddle River, NJ, EE. UU.: Pearson Prentice Hall, 2007.

[5] Python Software Foundation, “threading — Thread-based parallelism,” *Python 3 Documentation*. [En línea]. Disponible: https://docs.python.org/3/library/threading.html

[6] A. Silberschatz, P. B. Galvin y G. Gagne, *Operating System Concepts*, 10.ª ed. Hoboken, NJ, EE. UU.: Wiley, 2018.

[7] J. Oikarinen y D. Reed, “Internet Relay Chat Protocol,” IETF, RFC 1459, may. 1993. [En línea]. Disponible: https://www.rfc-editor.org/info/rfc1459

[8] Nmap Project, “Ncat — Netcat for the 21st Century.” [En línea]. Disponible: https://nmap.org/ncat/

[9] Python Software Foundation, “Socket Programming HOWTO,” *Python 3 Documentation*. [En línea]. Disponible: https://docs.python.org/3/howto/sockets.html

[10] R. C. Martin, *Agile Software Development: Principles, Patterns, and Practices*. Upper Saddle River, NJ, EE. UU.: Prentice Hall, 2003.

[11] A. S. Tanenbaum y D. J. Wetherall, *Computer Networks*, 5.ª ed. Boston, MA, EE. UU.: Pearson, 2011.

[12] T. Bray, Ed., “The JavaScript Object Notation (JSON) Data Interchange Format,” IETF, RFC 8259, dic. 2017. [En línea]. Disponible: https://www.rfc-editor.org/info/rfc8259

[13] “Python Prompt Toolkit 3.0 Documentation.” [En línea]. Disponible: https://python-prompt-toolkit.readthedocs.io/

[14] Python Software Foundation, “abc — Abstract Base Classes,” *Python 3 Documentation*. [En línea]. Disponible: https://docs.python.org/3/library/abc.html

[15] R. C. Martin, *Clean Code: A Handbook of Agile Software Craftsmanship*. Upper Saddle River, NJ, EE. UU.: Prentice Hall, 2008.

[16] Python Software Foundation, “logging — Logging facility for Python,” *Python 3 Documentation*. [En línea]. Disponible: https://docs.python.org/3/library/logging.html

[17] Python Software Foundation, “unittest — Unit testing framework,” *Python 3 Documentation*. [En línea]. Disponible: https://docs.python.org/3/library/unittest.html

[18] Microsoft, “Test-NetConnection,” *Microsoft Learn*. [En línea]. Disponible: https://learn.microsoft.com/en-us/powershell/module/nettcpip/test-netconnection

ANEXO A. CÓDIGO FUENTE COMPLETO

Se incluye el código completo del proyecto. Los archivos `app_servidor.py` y `chat/servidor.py` (servidor) y `app_cliente.py` y `chat/cliente.py` (cliente) corresponden a los entregables `servidor.py` y `cliente.py` del enunciado.

A.1 app_servidor.py

```
"""Punto de entrada del servidor. Uso: python app_servidor.py --host 0.0.0.0 --puerto 8080."""

import argparse
import logging

from chat.formato import FormateadorConsola
from chat.registros import configurar_logs
from chat.servidor import GestorChat, RegistroClientes, ServidorChat



def main() -> None:
    parser = argparse.ArgumentParser(description="Servidor del chat multiusuario LAN")
    parser.add_argument("--host", default="0.0.0.0", help="IP para escuchar; 0.0.0.0 habilita LAN")
    parser.add_argument("--puerto", type=int, default=8080, help="Puerto TCP (predeterminado: 8080)")
    argumentos = parser.parse_args()

    configurar_logs("servidor")
    gestor = GestorChat(RegistroClientes(), FormateadorConsola())
    try:
        ServidorChat(argumentos.host, argumentos.puerto, gestor).iniciar()
    except OSError as error:
        logging.getLogger(__name__).error("No se pudo iniciar el servidor: %s", error)
        raise SystemExit(1) from error



if __name__ == "__main__":
    main()
```

A.2 chat/servidor.py

```
"""Servidor TCP concurrente: conexiones, usuarios y difusión en LAN.

Responsabilidades separadas: RegistroClientes guarda las sesiones,
GestorChat realiza la difusión y ServidorChat acepta conexiones de red.
"""

import logging
import socket
import threading
from dataclasses import dataclass, field

from chat.formato import FormateadorMensaje
from chat.protocolo import codificar_evento, leer_evento, validar_alias, validar_texto

REGISTRO = logging.getLogger(__name__)



@dataclass(eq=False)
class SesionCliente:
    """Representa una conexión activa y sincroniza los envíos al socket."""

    conexion: socket.socket
    direccion: tuple
    alias: str
    candado_envio: threading.Lock = field(default_factory=threading.Lock)

    def enviar(self, paquete: bytes) -> None:
        """Impide mezclar bytes cuando varios hilos escriben al mismo cliente."""
        with self.candado_envio:
            self.conexion.sendall(paquete)

    def cerrar(self) -> None:
        """Interrumpe la conexión de una sesión que ya no responde."""
        try:
            self.conexion.shutdown(socket.SHUT_RDWR)
        except OSError:
            pass
        try:
            self.conexion.close()
        except OSError:
            pass



class RegistroClientes:
```

```
    """Gestiona de manera segura el conjunto de usuarios conectados."""

    def __init__(self) -> None:
        self._sesiones: dict[socket.socket, SesionCliente] = {}
        self._candado = threading.Lock()

    def registrar(self, sesion: SesionCliente) -> None:
        with self._candado:
            if any(otra.alias.casefold() == sesion.alias.casefold() for otra in self._sesiones.values()):
                raise ValueError("Ese alias ya está conectado. Escoge otro.")
            self._sesiones[sesion.conexion] = sesion

    def quitar(self, sesion: SesionCliente) -> bool:
        with self._candado:
            return self._sesiones.pop(sesion.conexion, None) is not None

    def obtener_todos(self) -> list[SesionCliente]:
        with self._candado:
            return list(self._sesiones.values())


class GestorChat:
    """Coordina mensajes sin controlar los sockets de escucha."""

    def __init__(self, registro: RegistroClientes, formateador: FormateadorMensaje) -> None:
        self._registro = registro
        self._formateador = formateador

    @property
    def registro(self) -> RegistroClientes:
        return self._registro

    def difundir(self, tipo: str, contenido: str) -> None:
        """Distribuye el evento a todos los clientes activos, incluido el emisor."""
        paquete = codificar_evento(tipo, contenido=contenido)
        for sesion in self._registro.obtener_todos():
            try:
                sesion.enviar(paquete)
            except OSError as error:
                REGISTRO.warning("Falló el envío a %s: %s", sesion.alias, error)
                # La lectura del hilo correspondiente terminará tras cerrar el socket.
                sesion.cerrar()

    def publicar_mensaje(self, alias: str, texto: str) -> None:
        contenido = self._formateador.formatear(alias, texto)
        self.difundir("mensaje", contenido)
        REGISTRO.info("Mensaje difundido | emisor=%s | longitud=%s", alias, len(texto))


class AtendedorCliente:
    """Se ocupa del ciclo de vida de un cliente particular."""

    def __init__(self, gestor: GestorChat) -> None:
        self._gestor = gestor

    def atender(self, conexion: socket.socket, direccion: tuple) -> None:
        sesion: SesionCliente | None = None
        REGISTRO.info("Conexión TCP desde %s:%s", *direccion[:2])
        try:
            with conexion, conexion.makefile("r", encoding="utf-8") as flujo:
                ingreso = leer_evento(flujo)
                if ingreso is None or ingreso.get("tipo") != "ingreso":
                    raise ValueError("El primer evento debe indicar el alias del usuario.")
                try:
                    alias = validar_alias(ingreso.get("alias"))
                    sesion = SesionCliente(conexion, direccion, alias)
                    self._gestor.registro.registrar(sesion)
                except ValueError as error:
                    # Enviamos el motivo al cliente antes de cerrar el socket.
                    conexion.sendall(codificar_evento("error", contenido=str(error)))
                    sesion = None
                    raise
                REGISTRO.info("Ingresó al chat | usuario=%s | ip=%s | puerto=%s", alias, *direccion[:2])
                self._gestor.difundir("sistema", f"{alias} se conectó al chat.")

                while True:
                    evento = leer_evento(flujo)
                    if evento is None or evento["tipo"] == "salida":
                        break
                    if evento["tipo"] != "mensaje":
                        raise ValueError("Tipo de evento inesperado.")
                    texto = validar_texto(evento.get("texto"))
                    self._gestor.publicar_mensaje(alias, texto)
        except (OSError, UnicodeError, ValueError) as error:
            REGISTRO.warning("Conexión %s:%s finalizó con aviso: %s", *direccion[:2], error)
```

```
        except Exception:
            REGISTRO.exception("Error inesperado con %s:%s", *direccion[:2])
        finally:
            if sesion is not None and self._gestor.registro.quitar(sesion):
                REGISTRO.info("Salió del chat | usuario=%s | ip=%s | puerto=%s", sesion.alias, *direccion[:2])
                self._gestor.difundir("sistema", f"{sesion.alias} salió del chat.")
            try:
                conexion.close()
            except OSError:
                pass


class ServidorChat:
    """Únicamente abre el puerto TCP y distribuye las conexiones entre hilos."""

    def __init__(self, host: str, puerto: int, gestor: GestorChat) -> None:
        self._host = host
        self._puerto = puerto
        self._atendedor = AtendedorCliente(gestor)

    def iniciar(self) -> None:
        """Acepta clientes hasta que se interrumpa el proceso con Ctrl+C."""
        with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as servidor:
            servidor.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
            servidor.bind((self._host, self._puerto))
            servidor.listen(20)
            REGISTRO.info("Servidor de chat activo en %s:%s", self._host, self._puerto)
            try:
                while True:
                    conexion, direccion = servidor.accept()
                    hilo = threading.Thread(
                        target=self._atendedor.atender,
                        args=(conexion, direccion),
                        name=f"cliente-{direccion[0]}-{direccion[1]}",
                        daemon=True,
                    )
                    hilo.start()
            except KeyboardInterrupt:
                REGISTRO.info("Servidor detenido mediante Ctrl+C.")
```

A.3 chat/formato.py

```
"""Presentación de mensajes separada de la comunicación por red (SOLID)."""

from abc import ABC, abstractmethod


class FormateadorMensaje(ABC):
    """Contrato que permite cambiar el formato sin modificar el servidor."""

    @abstractmethod
    def formatear(self, alias: str, texto: str) -> str:
        """Devuelve la representación visible de un mensaje de chat."""


class FormateadorConsola(FormateadorMensaje):
    """Presentación simple de una conversación por consola."""

    def formatear(self, alias: str, texto: str) -> str:
        return f"[{alias}]: {texto}"
```

A.4 chat/protocolo.py

```
"""Contrato de comunicación entre el servidor y los clientes.

Cada evento viaja como un objeto JSON codificado en UTF-8 y terminado en '\\n'.
El salto de línea permite distinguir mensajes aunque TCP junte varios paquetes.
"""

import json
import re
from typing import Any, TextIO

MAX_EVENTO_BYTES = 4096
MAX_TEXTO = 500
MAX_ALIAS = 20
PATRON_ALIAS = re.compile(r"^[\w -]+$", re.UNICODE)


def validar_alias(alias: str) -> str:
    """Valida y normaliza el nombre visible de una persona."""
    if not isinstance(alias, str):
        raise ValueError("El alias debe ser texto.")
```

```
    alias = alias.strip()
    if not 1 <= len(alias) <= MAX_ALIAS or not PATRON_ALIAS.fullmatch(alias):
        raise ValueError("Alias inválido: usa 1 a 20 caracteres (letras, números, espacios, - o _).")
    return alias


def validar_texto(texto: str) -> str:
    """Comprueba que el mensaje no esté vacío ni exceda el límite."""
    if not isinstance(texto, str):
        raise ValueError("El mensaje debe ser texto.")
    texto = texto.strip()
    if not 1 <= len(texto) <= MAX_TEXTO:
        raise ValueError("Escribe un mensaje de entre 1 y 500 caracteres.")
    return texto


def codificar_evento(tipo: str, **datos: Any) -> bytes:
    """Convierte un evento en una línea JSON lista para enviarse por TCP."""
    evento = {"tipo": tipo, **datos}
    paquete = (json.dumps(evento, ensure_ascii=False, separators=(",", ":")) + "\n").encode("utf-8")
    if len(paquete) > MAX_EVENTO_BYTES:
        raise ValueError("El evento supera el tamaño máximo permitido.")
    return paquete


def leer_evento(flujo: TextIO) -> dict[str, Any] | None:
    """Lee exactamente un evento; retorna None si se cierra la conexión.

    La lectura tiene límite para evitar consumir memoria sin control.
    """
    linea = flujo.readline(MAX_EVENTO_BYTES + 1)
    if not linea:
        return None
    if not linea.endswith("\n") or len(linea.encode("utf-8")) > MAX_EVENTO_BYTES:
        raise ValueError("Evento demasiado largo o sin terminador de línea.")
    try:
        evento = json.loads(linea)
    except json.JSONDecodeError as error:
        raise ValueError("El evento recibido no contiene JSON válido.") from error
    if not isinstance(evento, dict) or not isinstance(evento.get("tipo"), str):
        raise ValueError("El evento recibido no tiene un tipo válido.")
    return evento
```

A.5 chat/registros.py

```
"""Configuración uniforme de logs para el cliente y el servidor."""

import logging
import sys
from logging.handlers import RotatingFileHandler
from pathlib import Path


class ManejadorConsolaDinamico(logging.StreamHandler):
    """Envía logs al flujo actual, incluso si el prompt redirige stdout/stderr."""

    def emit(self, registro: logging.LogRecord) -> None:
        self.stream = sys.stderr
        super().emit(registro)


def configurar_logs(nombre_aplicacion: str) -> None:
    """Escribe logs en consola y archivo rotativo (máximo 1 MB por archivo)."""
    carpeta = Path(__file__).resolve().parent.parent / "logs"
    carpeta.mkdir(parents=True, exist_ok=True)

    formato = logging.Formatter(
        "%(asctime)s | %(levelname)-7s | %(threadName)s | %(name)s | %(message)s",
        datefmt="%Y-%m-%d %H:%M:%S",
    )
    raiz = logging.getLogger()
    raiz.setLevel(logging.INFO)
    raiz.handlers.clear()

    consola = ManejadorConsolaDinamico()
    consola.setFormatter(formato)
    raiz.addHandler(consola)

    archivo = RotatingFileHandler(
        carpeta / f"{nombre_aplicacion}.log",
        maxBytes=1_000_000,
        backupCount=3,
        encoding="utf-8",
    )
```

```
    archivo.setFormatter(formato)
    raiz.addHandler(archivo)
```

A.6 app_cliente.py

```
"""Punto de entrada del cliente. Usa prompt_toolkit para no interrumpir la escritura."""

import argparse
import logging

from prompt_toolkit import PromptSession
from prompt_toolkit.patch_stdout import patch_stdout

from chat.cliente import ClienteChat
from chat.protocolo import validar_alias
from chat.registros import configurar_logs



def main() -> None:
    parser = argparse.ArgumentParser(description="Cliente del chat multiusuario LAN")
    parser.add_argument("--host", default="127.0.0.1", help="IP real del equipo servidor en la LAN")
    parser.add_argument("--puerto", type=int, default=8080, help="Puerto TCP del servidor")
    argumentos = parser.parse_args()

    configurar_logs("cliente")
    sesion = PromptSession()
    try:
        alias = validar_alias(sesion.prompt("Tu nombre para el chat: "))
    except (ValueError, KeyboardInterrupt, EOFError) as error:
        print(f"No se inició el chat: {error}")
        return

    cliente = ClienteChat(argumentos.host, argumentos.puerto)
    try:
        cliente.conectar(alias)
        print("Conectado. Escribe y presiona Enter. Para salir: /salir")
        # patch_stdout hace que mensajes impresos desde el hilo de escucha
        # no borren la línea de texto que el usuario está escribiendo.
        with patch_stdout():
            while cliente.esta_activo():
                try:
                    texto = sesion.prompt("Tú > ")
                except (EOFError, KeyboardInterrupt):
                    break
                if texto.strip().lower() == "/salir":
                    break
                if not texto.strip():
                    continue
                try:
                    cliente.enviar(texto)
                except ValueError as error:
                    print(f"[AVISO] {error}")
                except OSError as error:
                    print(f"[ERROR] Se perdió la conexión: {error}")
                    break
                except ConnectionError as error:
                    print(f"[ERROR] {error}")
                    break
    except (OSError, ConnectionError) as error:
        logging.getLogger(__name__).error("No fue posible conectarse: %s", error)
        print("Comprueba la IP, el puerto, el firewall y que el servidor esté iniciado.")
    finally:
        cliente.desconectar()
        print("Chat finalizado.")



if __name__ == "__main__":
    main()
```

A.7 chat/cliente.py

```
"""Cliente concurrente: el usuario escribe mientras otro hilo recibe mensajes."""

import logging
import socket
import threading

from chat.protocolo import codificar_evento, leer_evento, validar_alias, validar_texto

REGISTRO = logging.getLogger(__name__)



class ClienteChat:
    """Conecta con el servidor, emite eventos y escucha sin bloquear el teclado."""
```

```
    def __init__(self, host: str, puerto: int) -> None:
        self._host = host
        self._puerto = puerto
        self._conexion: socket.socket | None = None
        self._activo = threading.Event()
        self._hilo_escucha: threading.Thread | None = None

    def conectar(self, alias: str) -> None:
        alias = validar_alias(alias)
        self._conexion = socket.create_connection((self._host, self._puerto), timeout=10)
        self._conexion.settimeout(None)
        self._activo.set()
        self._conexion.sendall(codificar_evento("ingreso", alias=alias))
        REGISTRO.info("Conectado al servidor %s:%s como %s", self._host, self._puerto, alias)
        self._hilo_escucha = threading.Thread(
            target=self._escuchar, name="escucha-mensajes", daemon=True
        )
        self._hilo_escucha.start()

    def _escuchar(self) -> None:
        assert self._conexion is not None
        try:
            with self._conexion.makefile("r", encoding="utf-8") as flujo:
                while self._activo.is_set():
                    evento = leer_evento(flujo)
                    if evento is None:
                        print("\n[INFO] El servidor cerró la conexión.")
                        break
                    if evento["tipo"] == "error":
                        print(f"\n[ERROR] {evento.get('contenido', 'Solicitud rechazada')}")
                        break
                    if evento["tipo"] in {"mensaje", "sistema"}:
                        print(f"\n{evento.get('contenido', '')}")
        except (OSError, UnicodeError, ValueError) as error:
            if self._activo.is_set():
                REGISTRO.warning("Se interrumpió la lectura: %s", error)
                print("\n[INFO] Se perdió la conexión.")
        finally:
            self._activo.clear()
            REGISTRO.info("Hilo de recepción terminado.")

    def enviar(self, texto: str) -> None:
        if not self._activo.is_set() or self._conexion is None:
            raise ConnectionError("No hay conexión activa con el servidor.")
        texto = validar_texto(texto)
        self._conexion.sendall(codificar_evento("mensaje", texto=texto))
        REGISTRO.info("Mensaje enviado | longitud=%s", len(texto))

    def esta_activo(self) -> bool:
        return self._activo.is_set()

    def desconectar(self) -> None:
        if self._conexion is None:
            return
        if self._activo.is_set():
            try:
                self._conexion.sendall(codificar_evento("salida"))
            except OSError:
                pass
        self._activo.clear()
        try:
            self._conexion.shutdown(socket.SHUT_RDWR)
        except OSError:
            pass
        self._conexion.close()
        REGISTRO.info("Conexión local cerrada.")
```

A.8 pruebas/test_chat.py

```
"""Pruebas unitarias y de integración mediante sockets reales en localhost."""

import io
import socket
import subprocess
import sys
import time
import unittest
from pathlib import Path

from chat.formato import FormateadorConsola
from chat.protocolo import codificar_evento, leer_evento, validar_alias, validar_texto

RAIZ = Path(__file__).resolve().parent.parent
```

```
class PruebasUnitarias(unittest.TestCase):
    def test_formato(self):
        self.assertEqual(FormateadorConsola().formatear("Ana", "Hola"), "[Ana]: Hola")

    def test_dos_eventos_consecutivos_en_tcp(self):
        contenido = codificar_evento("mensaje", texto="uno") + codificar_evento("mensaje", texto="dos")
        flujo = io.StringIO(contenido.decode("utf-8"))
        self.assertEqual(leer_evento(flujo)["texto"], "uno")
        self.assertEqual(leer_evento(flujo)["texto"], "dos")

    def test_validacion(self):
        self.assertEqual(validar_alias("  Ana  "), "Ana")
        self.assertEqual(validar_texto(" Hola "), "Hola")
        with self.assertRaises(ValueError):
            validar_alias("nombre_con_mas_de_20_caracteres")
        with self.assertRaises(ValueError):
            validar_texto(" ")


class PruebasIntegracion(unittest.TestCase):
    @classmethod
    def setUpClass(cls):
        with socket.socket() as temporal:
            temporal.bind(("127.0.0.1", 0))
            cls.puerto = temporal.getsockname()[1]
        cls.proceso = subprocess.Popen(
            [sys.executable, "app_servidor.py", "--host", "127.0.0.1", "--puerto", str(cls.puerto)],
            cwd=RAIZ,
            stdout=subprocess.DEVNULL,
            stderr=subprocess.DEVNULL,
        )
        for _ in range(50):
            try:
                with socket.create_connection(("127.0.0.1", cls.puerto), timeout=0.2):
                    return
            except OSError:
                time.sleep(0.05)
        cls.proceso.terminate()
        raise RuntimeError("El servidor no inició para las pruebas.")

    @classmethod
    def tearDownClass(cls):
        cls.proceso.terminate()
        try:
            cls.proceso.wait(timeout=3)
        except subprocess.TimeoutExpired:
            cls.proceso.kill()
            cls.proceso.wait()

    def test_dos_usuarios_conversan_y_se_desconectan(self):
        with socket.create_connection(("127.0.0.1", self.puerto), timeout=3) as uno, \
                socket.create_connection(("127.0.0.1", self.puerto), timeout=3) as dos:
            uno.settimeout(3)
            dos.settimeout(3)
            with uno.makefile("r", encoding="utf-8") as leer_uno, \
                    dos.makefile("r", encoding="utf-8") as leer_dos:
                uno.sendall(codificar_evento("ingreso", alias="Ana"))
                self.assertEqual(leer_evento(leer_uno)["tipo"], "sistema")
                dos.sendall(codificar_evento("ingreso", alias="Luis"))
                self.assertIn("Luis se conectó", leer_evento(leer_uno)["contenido"])
                self.assertEqual(leer_evento(leer_dos)["tipo"], "sistema")

                uno.sendall(codificar_evento("mensaje", texto="Hola a todos"))
                self.assertEqual(leer_evento(leer_uno)["contenido"], "[Ana]: Hola a todos")
                self.assertEqual(leer_evento(leer_dos)["contenido"], "[Ana]: Hola a todos")

                dos.sendall(codificar_evento("mensaje", texto="Hola, Ana"))
                self.assertEqual(leer_evento(leer_uno)["contenido"], "[Luis]: Hola, Ana")
                self.assertEqual(leer_evento(leer_dos)["contenido"], "[Luis]: Hola, Ana")

                dos.sendall(codificar_evento("salida"))
                self.assertIn("Luis salió", leer_evento(leer_uno)["contenido"])
                uno.sendall(codificar_evento("salida"))

    def test_cierre_imprevisto_de_un_cliente(self):
        with socket.create_connection(("127.0.0.1", self.puerto), timeout=3) as uno:
            uno.settimeout(3)
            with uno.makefile("r", encoding="utf-8") as lectura:
                uno.sendall(codificar_evento("ingreso", alias="Carla"))
                self.assertEqual(leer_evento(lectura)["tipo"], "sistema")
                dos = socket.create_connection(("127.0.0.1", self.puerto), timeout=3)
                dos.sendall(codificar_evento("ingreso", alias="Pedro"))
                self.assertIn("Pedro se conectó", leer_evento(lectura)["contenido"])
                dos.shutdown(socket.SHUT_RDWR)
                dos.close()  # Sin evento de salida: simula cierre imprevisto.
                self.assertIn("Pedro salió", leer_evento(lectura)["contenido"])
                uno.sendall(codificar_evento("mensaje", texto="Seguimos conectados"))
```

```
                self.assertEqual(leer_evento(lectura)["contenido"], "[Carla]: Seguimos conectados")
                uno.sendall(codificar_evento("salida"))

    def test_alias_duplicado_se_rechaza(self):
        with socket.create_connection(("127.0.0.1", self.puerto), timeout=3) as uno, \
                socket.create_connection(("127.0.0.1", self.puerto), timeout=3) as dos:
            uno.settimeout(3)
            dos.settimeout(3)
            with uno.makefile("r", encoding="utf-8") as lectura_uno, \
                    dos.makefile("r", encoding="utf-8") as lectura_dos:
                uno.sendall(codificar_evento("ingreso", alias="Marta"))
                self.assertEqual(leer_evento(lectura_uno)["tipo"], "sistema")
                dos.sendall(codificar_evento("ingreso", alias="Marta"))
                evento = leer_evento(lectura_dos)
                self.assertEqual(evento["tipo"], "error")
                self.assertIn("alias", evento["contenido"])
                uno.sendall(codificar_evento("salida"))


if __name__ == "__main__":
    unittest.main()
```

A.9 requirements.txt

```
# Mantiene el texto escrito mientras llegan mensajes de otros usuarios.
prompt_toolkit>=3.0,<4.0
```

ANEXO C. ACTA DE REUNIÓN DEL EQUIPO

Borrador redactado por el documentador para la revisión del líder. Los datos entre corchetes deben confirmarse.

Campo	Detalle
Proyecto	Taller Grupal Sockets Multihilo: Sistema de Chat Multiusuario en Red LAN por Consola
Asignatura y docente	Sistemas Distribuidos — Ing. Ricardo Proaño
Tipo de reunión	Integración y pruebas finales en red LAN
Fecha	16 de octubre del 2026
Hora	Aproximadamente de 17:20 a 17:45 (según los registros de las pruebas)
Lugar y modalidad	Presencial, red LAN 172.17.44.0/24 (equipos 172.17.44.82, .73 y .74)
Preside	Jairo Alejandro Ojeda (líder del equipo)
Redacta	Juan David Figueroa (documentador)

Asistente	Rol	Asistencia
Jairo Alejandro Ojeda	Líder y desarrollador del servidor	Presente
Josué Samuel Vele	Desarrollador del cliente	Presente
Juan David Figueroa	Documentador	Presente
Usuarios de prueba “Pedro” y “Tiago Daniel”	Participantes en la prueba con cuatro usuarios	Presentes

C.1 Orden del día

1) Verificación de los entregables de cada rol.
2) Configuración de la red LAN y comprobación de conectividad.
3) Ejecución de las pruebas con dos y con cuatro usuarios.
4) Revisión de evidencias, registros y hallazgos.
5) Asignación de pendientes y cierre.

C.2 Desarrollo

1) El líder inició el servidor en 172.17.44.82 con `--host 0.0.0.0 --puerto 8080` y confirmó el registro de arranque.
2) Los desarrolladores comprobaron la conectividad con `Test-NetConnection` desde 172.17.44.73 y 172.17.44.74 (resultado True).
3) Se probó la conversación entre dos clientes remotos, las salidas con `/salir`, la reconexión con el mismo alias y la participación del cliente local del anfitrión.
4) Se realizó la prueba con cuatro usuarios simultáneos (dos por equipo) y se revisó `servidor.log`.
5) Se detectó que, al cerrar el cliente, Python 3.14.5 muestra un aviso (`Fatal Python error: _enter_buffered_busy`) sin efectos sobre el servidor.
6) Se observó un desfase de unos cinco segundos entre los relojes de los equipos al comparar los registros.

C.3 Acuerdos

1) Dar por superadas las pruebas funcionales en LAN y adjuntar las capturas ordenadas cronológicamente al documento IEEE.
2) Registrar el aviso final del cliente como mejora pendiente y documentarlo en la discusión.
3) Integrar las ramas de funcionalidad en `develop`, validar y fusionar en `main` con una etiqueta de versión.
4) Entregar el documento IEEE, el README y los anexos el 16 de octubre del 2026.

C.4 Tareas pendientes

Tarea	Responsable	Plazo
Corregir el cierre del hilo de recepción (`join` en `desconectar`)	Josué Samuel Vele	**[REVISAR: definir plazo]**
Fusionar `develop` en `main` y crear la etiqueta `v1.0`	Jairo Alejandro Ojeda	**[REVISAR: definir plazo]**
Subir README, documento IEEE y carpeta `docs/capturas`	Juan David Figueroa	16/10/2026
Exportar el historial de GitHub para el Anexo E	Juan David Figueroa	16/10/2026

ANEXO D. BITÁCORA DEL LÍDER

Líder: Jairo Alejandro Ojeda. **Proyecto:** Taller Grupal Sockets Multihilo. Las fechas entre corchetes deben completarse; las del 16/10/2026 provienen de las marcas de tiempo de los registros.

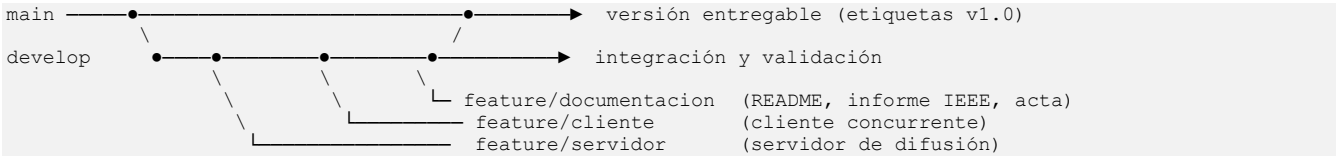
N.º	Fecha	Actividad	Responsable	Resultado
1	[fecha]	Análisis del enunciado y del servidor de eco; reparto de roles	Equipo	Roles y entregables definidos
2	[fecha]	Creación del repositorio y de las ramas `main`, `develop` y `feature/*`	Jairo	Flujo de ramas operativo
3	[fecha]	Diseño del protocolo de eventos JSON y de las clases	Equipo	Contrato servidor-cliente acordado
4	[fecha]	Implementación del servidor de difusión (`feature/servidor`)	Jairo	Hilo por cliente, registro y difusión
5	[fecha]	Implementación del cliente concurrente (`feature/cliente`)	Josué	Hilo de escucha y entrada con prompt_toolkit
6	[fecha]	Logs y pruebas automatizadas (`feature/logs-pruebas`)	Equipo	Logs rotativos y seis pruebas
7	16/10 17:24	Intento de conexión con el servidor aún apagado	Juan	Error controlado con mensaje orientativo
8	16/10 17:25	Inicio del servidor en 0.0.0.0:8080; cliente local a las 17:28	Jairo	Servidor activo
9	16/10 17:29–17:31	Verificación TCP con `Test-NetConnection`	Todos	Resultado True desde .73 y .74
10	16/10 17:32–17:39	Conversación remota, salidas y reconexiones	Juan y Josué	Mensajes, avisos y alias reutilizado
11	16/10 17:39	Mensaje del cliente local hacia la LAN	Jairo	Recibido en el equipo de Juan
12	16/10 17:40–17:41	Prueba con cuatro usuarios simultáneos	Todos	Difusión y salidas correctas
13	16/10	Revisión de hallazgos	Jairo	Aviso final de Python anotado como pendiente
14	[fecha]	Integración en `develop` y `main`, etiqueta y entrega	Jairo	[completar]

Decisiones del líder: (1) arquitectura de un hilo por conexión por su claridad didáctica; (2) protocolo JSON por línea con límites de tamaño; (3) no registrar el contenido de los mensajes, solo su longitud; (4) inyección del formateador para cumplir SOLID.

Riesgos y observaciones: conectividad y cortafuegos en la LAN; relojes desincronizados entre equipos; aviso de Python al cerrar el cliente. **Próximos pasos:** corregir el cierre del hilo, fusionar en `main` y etiquetar la versión.

ANEXO E. CONTROL DE VERSIONES E HISTORIAL DE GITHUB

E.1 Estrategia de ramas



Rama	Propósito	Responsable
`main`	Versión estable y entregable	Líder
`develop`	Integración de las funcionalidades terminadas	Líder
`feature/servidor`	Servidor de difusión	Jairo Alejandro Ojeda
`feature/cliente`	Cliente concurrente con hilos	Josué Samuel Vele
`feature/documentacion`	README, documento IEEE, acta y control de versiones	Juan David Figueroa
`feature/logs-pruebas`	Logs y pruebas en LAN (rama adicional visible en las evidencias)	Equipo

E.2 Flujo de trabajo y convención de commits

```
git switch -c feature/servidor develop      # crear la rama desde develop
git add chat/servidor.py app_servidor.py
git commit -m "feat(servidor): difundir mensajes a todos los clientes"
git push -u origin feature/servidor          # abrir Pull Request hacia develop

git switch develop && git merge --no-ff feature/servidor
git switch main    && git merge --no-ff develop
git tag -a v1.0 -m "Entrega final del taller"
git push origin main develop --follow-tags
```

Prefijos de commit: `feat`, `fix`, `docs`, `test` y `refactor`. Todo cambio hacia `develop` se integra mediante Pull Request revisado por el líder.

E.3 Ramas observadas en las evidencias

1) `feature/servidor`: rama activa en el IDE del equipo anfitrión durante las pruebas (visible en las capturas originales del IDE).
2) `feature/cliente`: creada con `git switch -c feature/cliente` en el equipo de Josué (Fig. 5b).
3) `feature/logs-pruebas`: creada con `git switch -c feature/logs-pruebas` en el equipo de Juan (Fig. 5a) y usada en las pruebas de cuatro usuarios.